



Rapport : Méthodes directes et itératives pour résoudre l'équation de la chaleur 1D

M1 CHPS : Calcul Numérique

LAKHANI Naila

Janvier 2026

Matériel

Hardware

- Modèle : Lenovo IdeaPad Slim 3 (ordinateur portable)
- Architecture : x86_64
- Processeur : AMD Ryzen 5 7520U with Radeon Graphics
- Fréquence du processeur : 4,3 GHz
- RAM : 16 Go
- Cœurs : 4
- Threads : 8 (2 par cœur)
- Cache :
 - L1d : 128 Kb (par cœur)
 - L1i : 128 Kb (par cœur)
 - L2 : 2 Mb (par cœur)
 - L3 : 4 Mb

Software

- Compilateur : gcc (GCC) 15.2.1 20250808 (Red Hat 15.2.1-1)

Introduction

Nous nous intéressons à **résoudre un système linéaire** de l'équation de la chaleur 1D stationnaire à l'aide des outils vu en cours de **Calcul Numérique**. L'implémentation est en **langage C** avec les libraires **BLAS** et **LAPACK**. Le TP contient deux grandes parties : une résolution **directe** et résolution **itérative**, dans le but de comparer ces deux méthodes. Ce rapport à pour but de répondre aux questions des exercices **5** et **6** du TP. Le reste étant contenu dans le code complété disponible sur le lien ci-dessous :

https://github.com/lakhss/cn_lakhani

Exercice 5

Question 5.1

Cf dépôt Github.

Question 5.2

Fonction	Temps d'exécution (secondes)
dgbtrf	0.00003
dgbtrs	0.000003
dgbsv	0.000031

Table 1: Temps d'exécution des différentes fonctions en utilisant LAPACK et BLAS.

Exercice 6

Question 6.1

Cf dépôt Github.

Question 6.2

Un bon moyen de valider notre implémentation est de comparer l'erreur relative lorsque l'on utilise notre implémentation et lorsque l'on utilise la factorisation LU de LAPACK (`dgbtrf_`), on trouve la même valeur

$$relres = 2.692638e - 16 \quad (1)$$

donc notre implémentation est correcte. En plus de cela, on vérifie de manière "brute" en implémentant des `printf` pour pouvoir afficher la matrice AB avant et après factorisation, voilà ce que l'on obtient en lançant le docker :

```
----- value of AB initially -----
0.000000 0.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 -1.000000
0.000000 -1.000000 2.000000 0.000000

[ Exécution de la méthode ]

----- value of AB after factorization -----
0.000000 0.000000 2.000000 -0.500000
0.000000 -1.000000 1.500000 -0.666667
0.000000 -1.000000 1.333333 -0.750000
```

```

0.000000 -1.000000 1.250000 -0.800000
0.000000 -1.000000 1.200000 -0.833333
0.000000 -1.000000 1.166667 -0.857143
0.000000 -1.000000 1.142857 -0.875000
0.000000 -1.000000 1.125000 0.000000

```

Ce qui nous confirme encore une fois que la factorisation est correcte.

Question 6.3

Fonction	Temps d'exécution (secondes)
dgbtrf	0.000001
dgbtrs	0.000024
dgbsv	0.000031

Table 2: Temps d'exécution des différentes fonctions en utilisant notre propre implémentation de dgbtrf.

Méthode	Temps d'exécution (secondes)
TRF	0.000033
TRI	0.000025
SV	0.000031

Table 3: Temps d'exécution des différentes méthodes directes de résolution de Poisson 1D.

La résolution du système est plus rapide en utilisant notre implémentation de factorisation LU et dgbtrs que en utilisant dgbtrf et dgbtrs ou en utilisant directement dgbsv. On s'attendait à ce que dgbsv soit plus rapide que la combinaison de dgbtrf + dgbtrs, cependant c'est assez étonnant de voir que notre implémentation est la plus rapide car le but de LAPACK et de BLAS est de résoudre des équations linéaires de manière optimisée.

Conclusion

Nous avons appliqué les algorithmes vu en cours à l'aide d'outils comme **Docker** et **Makefile** via un exemple concret qui est la résolution de l'équation **Poisson 1D**. Nous avons pu comparer les performances des bibliothèques **LAPACK** et **BLAS** par rapport à celles de nos propres implémentations et grâce à cela pu se rendre compte du fonctionnement de ces bibliothèques. Cependant, les résultats ne sont pas en accord avec les résultats attendus, nous nous attendions à voir des meilleurs performances en utilisant **LAPACK** et **BLAS**.